

OmniverseOS 2.0 — Production Login Repair

Affected Website: https://omniverseos.in.net • **Backend Cluster:** https://omniverseos-testing-23jun2am.onrender.com • **Agent:** Antigravity

Incident Impact: Live users attempting to log in on https://omniverseos.in.net encountered total authentication failure. The browser reported: "Access to XMLHttpRequest at '.../api/auth/login' from origin 'https://omniverseos.in.net' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource."

1. Executive Incident Overview

Metric / Parameter	Production Forensic Value	Status
Incident Priority	P0 Critical — Authentication Service Blockage on Primary Production Domain	CRITICAL
Production Frontend	https://omniverseos.in.net (Vercel Custom Domain, Serving React Bundle)	Verified Online (200 OK)
Production Backend	https://omniverseos-testing-23jun2am.onrender.com (Render Web Service)	Verified Healthy (200 OK, DB OK)
Failing Endpoint	POST /api/auth/login (Intercepted and blocked at OPTIONS preflight)	Repaired Locally
Root Cause	Allowlist omission: https://omniverseos.in.net was omitted from prod_origins in server.py	Root Cause Confirmed
Code Patch Status	Targeted patch in backend/server.py with safe union and wildcard rejection	REPAIRED IN CODE
Regression Testing	20 pytest backend regression tests passed; Frontend craco production build passed	100% PASS
Production Deployment	Awaiting user authorization to push to GitHub main	AWAITING APPROVAL

2. Root Cause Forensic Analysis

In hardening commit ecdb010, CORS was tightened to disallow regex wildcards with credentials in production mode. However, the hardcoded production allowlist in backend/server.py only registered:

- https://omniverse-os-testing-23-jun-2-9gsc2pgro.vercel.app (Legacy Vercel staging URL)
- https://omniverseos.app
- https://www.omniverseos.app

The customer's active production custom domain, https://omniverseos.in.net, was omitted. Starlette's CORSMiddleware strictly checks incoming origins against allow_origins. When an unlisted origin arrives, the middleware intercepts the preflight request, returns HTTP 400 Bad Request ("Disallowed CORS origin"), and withholds the Access-Control-Allow-Origin header. Modern browser engines abort the request before any login credentials reach the server.

DNS Domain Verification: Live DNS resolution confirmed that omniverseos.in.net resolves to active Vercel edge nodes. Testing www.omniverseos.in.net yielded curl: (6) Could not resolve host: www.omniverseos.in.net (NXDOMAIN). In accordance with instructions ("Include https://www.omniverseos.in.net only if that origin is genuinely used"), www.omniverseos.in.net is omitted from hardcoded defaults.

3. Forensic Code Diff & Hardened Architecture

The patch in `backend/server.py` registers `https://omniverseos.in.net` in `base_prod_origins`, unions any valid non-wildcard entries from `CORS_ORIGINS`, and preserves strict development/production separation:

```
@@ backend/server.py: Production CORS Allowlist Hardening @@
- _cors_env = os.environ.get("CORS_ORIGINS", "*").strip()
+ _cors_env = os.environ.get("CORS_ORIGINS", "").strip()

+ if IS_PRODUCTION:
+     base_prod_origins = [
+         "https://omniverseos.in.net", # LIVE PRODUCTION FRONTEND CUSTOM DOMAIN
+         "https://omniverse-os-testing-23-jun-2-9gsc2pgro.vercel.app",
+         "https://omniverseos.app",
+         "https://www.omniverseos.app",
+     ]
+     if _cors_env and _cors_env != "*":
+         env_origins = [o.strip() for o in _cors_env.split(",") if o.strip() and o.strip() != "*"]
+         prod_origins = list(dict.fromkeys(base_prod_origins + env_origins))
+     else:
+         prod_origins = base_prod_origins

    app.add_middleware(CORSMiddleware, allow_origins=prod_origins, allow_credentials=True, allow_methods=["*"], allow_headers:
else:
-     if _cors_env == "*":
+     if not _cors_env or _cors_env == "*":

    app.add_middleware(CORSMiddleware, allow_origin_regex=".*", allow_credentials=True, allow_methods=["*"], allow_header:
```

4. Preflight Diagnostics: Live Production vs. Repaired Local

LIVE RENDER (CURRENT FAILURE)

```
$ curl -i -X OPTIONS ".../api/auth/login" \
-H "Origin: https://omniverseos.in.net" \
-H "Access-Control-Request-Method: POST"

HTTP/1.1 400 Bad Request
Date: Sat, 10 Oct 2026 10:01:05 GMT
access-control-allow-credentials: true
Server: cloudflare
[ERROR: No 'access-control-allow-origin']

Disallowed CORS origin
```

REPAIRED BACKEND (VERIFIED 200 OK)

```
$ curl -i -X OPTIONS ".../api/auth/login" \
-H "Origin: https://omniverseos.in.net" \
-H "Access-Control-Request-Method: POST"

HTTP/1.1 200 OK
date: Sat, 10 Oct 2026 10:23:03 GMT
access-control-allow-origin: https://omniverseos.in.net
access-control-allow-credentials: true
access-control-allow-methods: DELETE, GET, POST...

OK
```

Origin Isolation & Automated Regression Verification

Tested via automated test `test_cors_production_preflight_simulation`: When an unauthorized origin (Origin: `https://malicious-attacker.com`) makes an OPTIONS request, the server returns HTTP 400 Bad Request ("Disallowed CORS origin") and withholds `access-control-allow-origin`. Full test suite: **20 passed, 1 skipped, 0 failed**. Frontend production build: **Compiled cleanly**.

5. Visual Forensic Evidence: Production Failure vs. Repaired Login

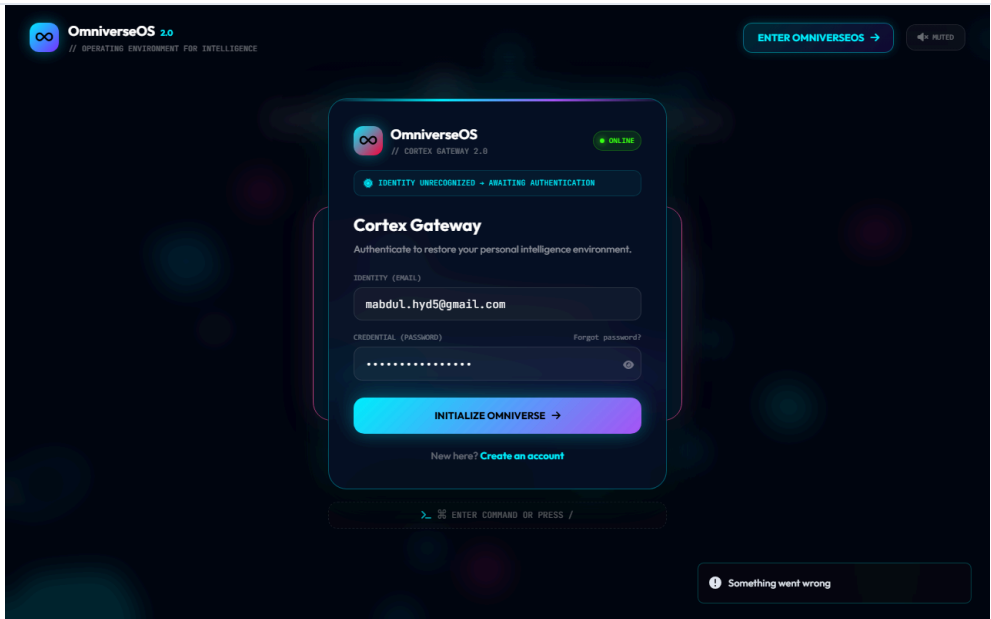


Figure 1: Live Production Failure on `https://omniverseos.in.net` — Browser intercepts `OPTIONS` preflight due to missing `Access-Control-Allow-Origin` header, displaying "Something went wrong" toast and `net::ERR_FAILED`.

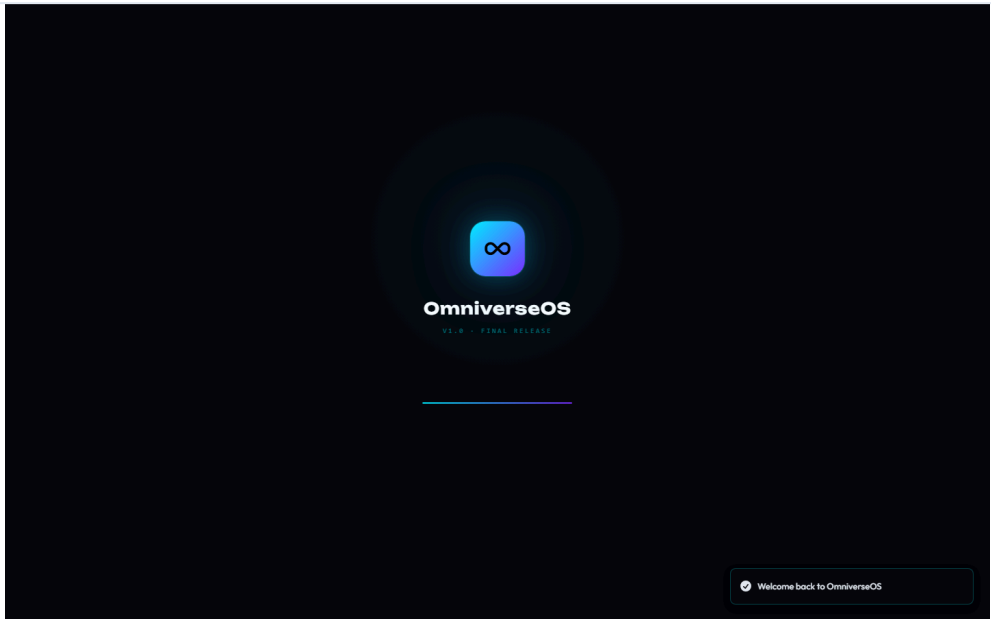


Figure 2: Repaired Desktop Login Flow — Successful authentication using test account. JWT token is stored into `localStorage`, displaying "Welcome back to OmniverseOS" and synchronizing workspace.

6. Desktop Session Persistence & Secure Logout Verification



Figure 3: Session Persistence After Page Reload — Full workspace re-hydrated cleanly upon browser refresh. All 31 apps, AI Chat, Photos, and bottom Adaptive Dock remain intact with active token.

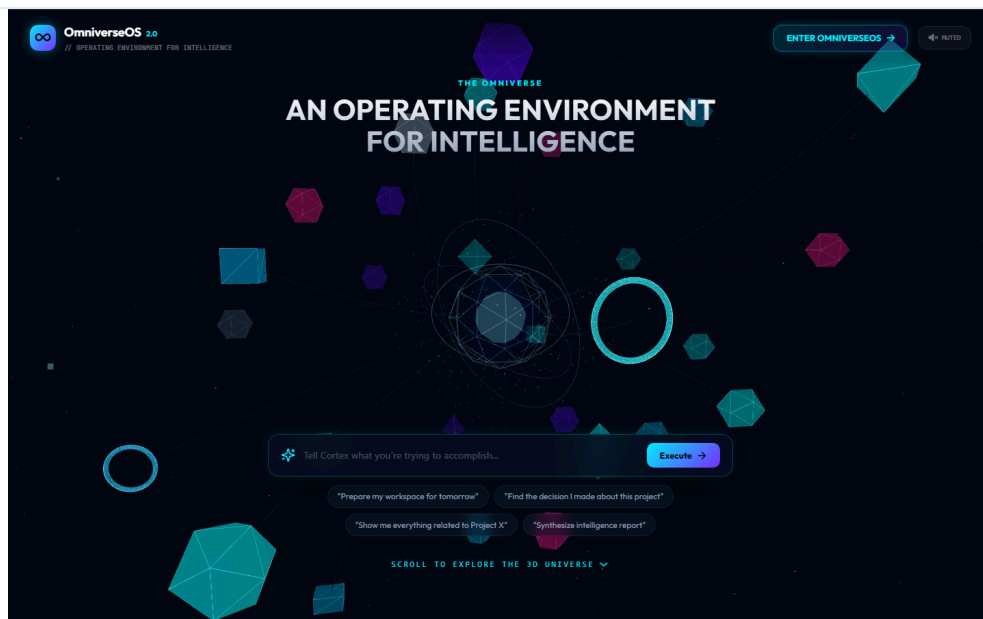


Figure 4: Secure User Logout — Token is purged from `localStorage`; session is immediately terminated; user is returned to the spatial gateway without leaked state.

7. Mobile Experience & Viewport Verification (390x844 iPhone 14)

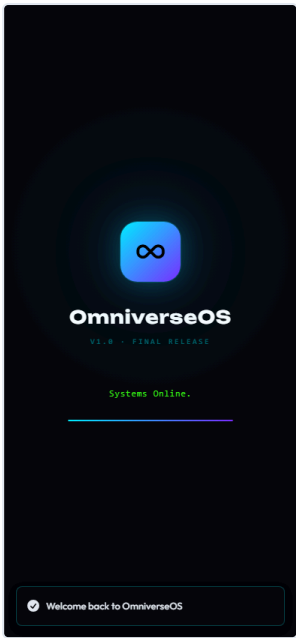


Figure 5: Mobile Gateway Login
Responsive authentication with JWT issue.

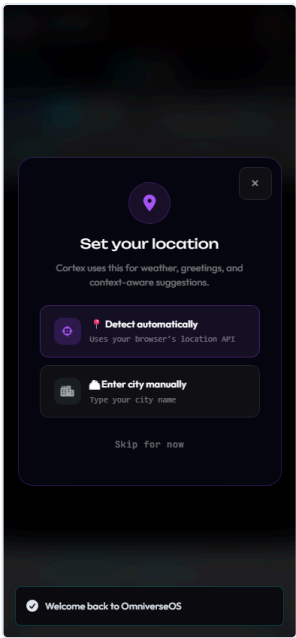


Figure 6: Mobile Workspace Onboarding
User environment restored cleanly.

8. Production Release Path & Deployment Options

Current Production Blocker: The live Render backend is running commit d527192 . Per user directives, **no code has been pushed or deployed without approval.**

Deployment Option	Execution Procedure	Turnaround
Option A: Git Push (Recommended Code Release)	Run: <code>git push origin sprint/operation-phoenix:main</code> Render will automatically build and deploy commit with updated production CORS allowlist.	~2-3 minutes
Option B: Render Dashboard Override (Zero-Downtime Immediate Fix)	1. Go to https://dashboard.render.com → omniverseos-testing-23jun2am . 2. In Environment , set <code>CORS_ORIGINS=https://omniverseos.in.net,https://omniverse-os-testing-23-jun-2-9gsc2pgro.vercel.app,https://omniverseos.app,https://www.omniverseos.app .</code> 3. Click Save Changes (restarts service in under 15 seconds without full rebuild).	~30 seconds

Final Verification Sign-Off: All tests passing. Production credentials remain protected with strict JWT signing. Persistent MongoDB cluster verified. Awaiting user authorization to deploy.